Kubernetes — Complete Knowledge Refresher (Java Full Stack Engineer)

> Everything you need to know about K8s — architecture, objects, networking, storage, security, scaling, Java-specific concerns, and production operations. Interview-calibrated for 7+ YOE.

Table of Contents

1. What is Kubernetes & Why It Exists
2. Architecture — The Full Picture
3. Core Objects — Workloads
4. Networking — How Traffic Flows
5. Storage — Persistence in K8s
6. Configuration — ConfigMaps, Secrets, Env
7. Scheduling — Where Pods Land
8. Scaling — HPA, VPA, Cluster Autoscaler
9. Security — RBAC, Pod Security, Secrets
10. Observability — Probes, Logs, Monitoring
11. Deployment Strategies — Zero Downtime
12. Helm — Package Management
13. Java on Kubernetes — JVM-Specific Concerns
14. Namespaces, Resource Quotas & Limit Ranges
15. CRDs & Operators
16. Service Mesh (Istio Basics)
17. CI/CD with Kubernetes
18. Troubleshooting Playbook
19. Essential Commands Cheat Sheet

---

1. What is Kubernetes & Why It Exists

Kubernetes (K8s) is a **container orchestration platform**. It automates deployment, scaling, networking, and management of containerized applications.

Before K8s:

```
Problem: You have 50 microservices, each in Docker containers.
  - Which server does each container run on?
  - What if a server dies? Who restarts the containers?
  - How do containers find each other (service discovery)?
  - How do you scale from 3 to 30 replicas under load?
  - How do you roll out a new version without downtime?
  - How do you manage secrets, configs, storage?

Answer: You do it all manually. Or you build K8s.
```

What K8s solves:

Concern	K8s Solution
Where to run containers	Scheduler places Pods on nodes
Self-healing	Restarts crashed containers, replaces dead nodes
Service discovery	DNS + Services
Load balancing	kube-proxy + Services
Scaling	HPA, VPA, Cluster Autoscaler
Rolling updates	Deployment controller
Config management	ConfigMaps + Secrets
Storage	PersistentVolumes
Access control	RBAC

As a Java dev, you care because: Your Spring Boot / Quarkus / Micronaut apps run in containers, deployed to K8s clusters in production. You write Dockerfiles, Helm charts, K8s manifests, and configure probes, resources, and environment. K8s is your runtime.

---

2. Architecture — The Full Picture

A K8s cluster has two planes: **Control Plane** (brain) and **Worker Nodes** (muscle).

```
┌──────────────────────── CONTROL PLANE ────────────────────────┐
│                                                                │
│  ┌──────────────┐  ┌──────────────────┐  ┌────────────────┐   │
│  │ kube-apiserver│  │ kube-controller  │  │ kube-scheduler│   │
│  │              │  │    manager       │  │               │   │
│  └──────┬───────┘  └──────────────────┘  └────────────────┘   │
│         │                                                      │
```

```
│         │          ┌──────────────────┐                    │
│         └──────────│      etcd        │                    │
│                    │ (key-value store) │                    │
│                    └──────────────────┘                    │
│                                                            │
│  ┌──────────────────┐ (optional)                          │
│  │ cloud-controller  │                                     │
│  │    manager        │                                     │
│  └──────────────────┘                                      │
└────────────────────────────────────────────────────────────┘


┌──────────────── WORKER NODE 1 ─────────────────┐
│  ┌──────────┐  ┌────────────┐  ┌────────────┐  │
│  │ kubelet  │  │ kube-proxy │  │ container  │  │
│  │          │  │            │  │  runtime   │  │
│  └──────────┘  └────────────┘  │(containerd)│  │
│                                └────────────┘  │
│  ┌────────┐ ┌────────┐ ┌────────┐             │
│  │ Pod A  │ │ Pod B  │ │ Pod C  │             │
│  └────────┘ └────────┘ └────────┘             │
└────────────────────────────────────────────────┘


┌──────────────── WORKER NODE 2 ─────────────────┐
│  (same structure: kubelet, kube-proxy, runtime)│
│  ┌────────┐ ┌────────┐                         │
│  │ Pod D  │ │ Pod E  │                         │
│  └────────┘ └────────┘                         │
└────────────────────────────────────────────────┘
```

2.1 Control Plane Components

kube-apiserver

```
Role: The FRONT DOOR of the cluster. Every operation goes through it.
  - kubectl commands → API server
  - Controllers → API server
  - Kubelet → API server
  - Dashboard/CI-CD → API server

What it does:
  1. Receives all REST API requests
  2. Authenticates the caller (certs, tokens, OIDC)
  3. Authorizes the request (RBAC)
  4. Runs Admission Controllers (mutate / validate)
```

```
  5. Persists the object to etcd
  6. Returns response to caller

Key fact: It's the ONLY component that talks to etcd directly.
         All other components talk to the API server.
```

etcd

```
Role: Distributed key-value store. THE source of truth for all cluster state.

What's stored:
  /registry/pods/default/my-pod         → Pod spec and status
  /registry/deployments/default/my-app  → Deployment definition
  /registry/services/default/my-svc     → Service definition
  /registry/secrets/default/db-pass     → Secret data

Key facts:
  - Uses Raft consensus (needs quorum: run 3 or 5 nodes in production)
  - Latency-sensitive — needs fast SSD disks
  - All data persisted here — lose etcd, lose the cluster
  - Take regular snapshots: etcdctl snapshot save backup.db
  - Only the API server reads/writes etcd

If etcd goes down:
  - Existing running containers KEEP running (kubelet is independent)
  - But no new deployments, no scaling, no healing, no scheduling
  - Cluster is effectively frozen until etcd recovers
```

kube-controller-manager

```
Role: Runs all the controllers — the reconciliation loops that make
      the ACTUAL state match the DESIRED state.

Controllers inside it (each runs as a goroutine):
  - Deployment Controller: watches Deployments → manages ReplicaSets
  - ReplicaSet Controller: watches ReplicaSets → creates/deletes Pods
  - Node Controller: monitors node health, marks NotReady, evicts Pods
  - Job Controller: watches Jobs → creates Pods for batch work
  - EndpointSlice Controller: watches Services → updates endpoint lists
  - ServiceAccount Controller: creates default SA for new namespaces
  - Namespace Controller: cleans up resources when namespace is deleted

How it works (reconciliation loop):
  while true:
```

```
    desired_state = read from etcd (via API server)
    actual_state  = observe current reality
    if actual != desired:
      take action to converge (create Pod, delete Pod, etc.)
    sleep(interval)

This is WHY K8s is declarative — you declare desired state, controllers converge.
```

kube-scheduler

```
Role: Decides WHICH node a new Pod runs on.

Only acts on Pods that have no nodeName assigned (Pending Pods).

Two phases:
  Phase 1: FILTERING (eliminate unsuitable nodes)
    - Enough CPU/memory?
    - Pod tolerates node's taints?
    - Node affinity matches?
    - PVC zone affinity?
    - Not cordoned/unschedulable?

  Phase 2: SCORING (rank surviving nodes)
    - LeastRequestedPriority (prefer emptier nodes)
    - BalancedResourceAllocation (even CPU/memory ratio)
    - InterPodAffinity (prefer co-location if specified)
    - TopologySpreadConstraints (spread across zones)

  Result: Pod.spec.nodeName = <best-scoring-node>

Does NOT run containers — that's kubelet's job.
```

cloud-controller-manager

```
Role: Integrates with cloud provider APIs (AWS/GCP/Azure).

Handles:
  - LoadBalancer Services → provisions cloud LB (ELB, ALB, etc.)
  - Node lifecycle → detects when a cloud VM is terminated
  - Route management → sets up cloud network routes for Pod CIDRs
  - Volume provisioning → creates cloud disks (EBS, GCE PD)
```

```
Only exists in cloud-managed clusters (EKS, GKE, AKS).
Not present in bare-metal or on-prem clusters.
```

2.2 Worker Node Components

kubelet

```
Role: The agent on every worker node. Makes sure containers are running.

What it does:
  1. Watches API server for Pods assigned to its node
  2. Pulls container images (via containerd / CRI-O)
  3. Creates containers via CRI (Container Runtime Interface)
  4. Sets up Pod networking (via CNI plugin)
  5. Mounts volumes (PVCs, ConfigMaps, Secrets)
  6. Runs health probes (liveness, readiness, startup)
  7. Reports Pod status back to API server
  8. Sends node heartbeats (every 10s via Lease objects)

Key fact: kubelet operates INDEPENDENTLY of the control plane.
  If the API server goes down, kubelet keeps running existing Pods.
  It just can't receive new work or report status.
```

kube-proxy

```
Role: Network proxy on every node. Implements Service networking.

When a Service is created:
  kube-proxy creates rules so that traffic to ServiceIP:Port
  gets load-balanced across the Service's backing Pods.

Three modes:
  iptables (default): writes iptables rules per Service/endpoint
    - Pros: stable, well-tested
    - Cons: slow with 5000+ Services (linear iptables scan)

  IPVS: uses Linux IPVS (IP Virtual Server) kernel module
    - Pros: faster with many Services (hash-based lookup)
    - Cons: needs IPVS kernel modules

  userspace (legacy): proxies in userspace — slow, deprecated

How it works:
```

```
  Service: my-api → ClusterIP 10.96.45.12:80
    Backend Pods: 10.244.1.5:8080, 10.244.2.8:8080

  kube-proxy creates iptables rules:
    Traffic to 10.96.45.12:80 →
      50% chance → DNAT to 10.244.1.5:8080
      50% chance → DNAT to 10.244.2.8:8080
```

Container Runtime

```
Role: Actually runs containers. K8s doesn't run containers itself.

K8s talks to the runtime via CRI (Container Runtime Interface).

Common runtimes:
  containerd — most common (used by Docker, EKS, GKE)
  CRI-O — lightweight, designed specifically for K8s (used by OpenShift)

Note: Docker as a runtime was REMOVED in K8s 1.24 (dockershim deprecated).
  Docker images still work — containerd pulls them.
  You just can't use Docker daemon as the K8s runtime anymore.
```

---

3. Core Objects — Workloads

3.1 Pod

```
The smallest deployable unit. One or more containers sharing:
  - Network namespace (same IP, same localhost)
  - Storage volumes
  - Lifecycle (created and destroyed together)

You almost NEVER create Pods directly.
You create Deployments/StatefulSets/Jobs that create Pods for you.

Multi-container Pod patterns:
  Sidecar:    helper container alongside main (Envoy proxy, log shipper)
  Init:       runs before main container (DB migration, config fetch)
  Ambassador: proxy container for external communication
  Adapter:    transforms output of main container

Pod lifecycle:
```

```
  Pending → Running → Succeeded/Failed
                ↓
          CrashLoopBackOff (container keeps crashing)
```

3.2 Deployment

```
Manages stateless applications. Most common workload object.

What it gives you:
  - Declarative updates (change image tag → rolling update happens)
  - Rolling updates with zero downtime
  - Rollback to previous versions
  - Scaling (replica count)

Hierarchy:
  Deployment → manages → ReplicaSet(s) → manages → Pod(s)

  Deployment: my-app
    └── ReplicaSet: my-app-7b4f9c (current, 3 Pods)
    └── ReplicaSet: my-app-5a2d8e (previous, 0 Pods, kept for rollback)

Update strategies:
  RollingUpdate (default):
    - maxUnavailable: 25% — at most 25% of Pods can be down
    - maxSurge: 25% — at most 25% extra Pods during rollout

  Recreate:
    - Kill all old Pods first, then create new ones
    - Has downtime — use only when you can't run two versions simultaneously
```

3.3 ReplicaSet

```
Ensures N identical Pods are running at all times.

You almost NEVER create ReplicaSets directly.
Deployments manage them for you and add rolling updates + rollback.

The only time to know about ReplicaSets:
  - Debugging: kubectl get rs shows current and old ReplicaSets
  - Understanding rollbacks: rollback = scale old RS up, new RS down
```

3.4 StatefulSet

```
For stateful applications where Pods need:
  - Stable network identity (db-0, db-1, db-2 — not random names)
  - Stable storage (each Pod gets its own PVC)
  - Ordered deployment and scaling

Key behaviors:
  - Pods created in order: db-0 first, then db-1, then db-2
  - Pods deleted in reverse: db-2 first, then db-1, then db-0
  - Each Pod gets a DNS name: db-0.db-headless.namespace.svc.cluster.local
  - Each Pod gets its own PVC (persists across Pod restarts)

When to use:
  - Databases (PostgreSQL, MySQL, MongoDB)
  - Message brokers (Kafka, RabbitMQ)
  - Distributed systems (ZooKeeper, etcd, Elasticsearch)
  - Anything that needs "I am node 0" identity

Requires: a Headless Service (ClusterIP: None) for DNS to work.
```

3.5 DaemonSet

```
Runs exactly ONE Pod per node (or per matching node).

When a new node joins: DaemonSet automatically schedules a Pod on it.
When a node is removed: Pod is garbage collected.

Use cases:
  - Log collection: Fluentd, Filebeat on every node
  - Monitoring: Datadog agent, Prometheus node-exporter
  - Networking: Calico, Cilium CNI on every node
  - Storage: CSI node plugins
  - kube-proxy itself runs as a DaemonSet

Targeting specific nodes:
  nodeSelector:
    gpu: "true"
  → DaemonSet only runs on nodes labeled gpu=true
```

3.6 Job & CronJob

```
Job: Run-to-completion workload. Pod runs, finishes, done.
  - Use for: batch processing, DB migrations, one-time tasks
  - completions: how many Pods must succeed
  - parallelism: how many run concurrently
  - backoffLimit: retry count before marking failed

CronJob: Creates Jobs on a schedule.
  - Use for: periodic backups, report generation, cleanup
  - schedule: "0 2 * * *" (cron syntax)
  - concurrencyPolicy: Allow / Forbid / Replace

Java relevance:
  - Spring Batch jobs running as K8s Jobs
  - Flyway/Liquibase DB migration as init container or Job
  - Scheduled report generation as CronJob
```

---

4. Networking — How Traffic Flows

4.1 The Three Rules

```
Rule 1: Every Pod gets its own unique IP address
Rule 2: All Pods can reach all other Pods (no NAT) — even across nodes
Rule 3: Agents on a node can reach all Pods on that node
```

4.2 CNI (Container Network Interface)

```
CNI plugins implement the actual Pod networking.
K8s delegates all networking to the CNI plugin.

Popular choices:
  Calico:   BGP-based routing, supports Network Policies, high performance
  Cilium:   eBPF-based, advanced security + observability, growing fast
  Flannel:  Simple VXLAN overlay, no Network Policy support
  Weave:    Mesh overlay, simple setup

Same-node Pod-to-Pod:
  Pod A → veth → bridge (cbr0) → veth → Pod B
```

```
Cross-node Pod-to-Pod:
  Pod A → veth → bridge → node routing → [overlay or BGP] → Node 2 → bridge →
veth → Pod B
```

4.3 Services

```
Services provide stable networking for a set of Pods.
Pods come and go (IPs change), Services don't.

Four types:

┌──────────────────────────────────────────────────────────────┐
│ ClusterIP (default)                                          │
│  - Internal virtual IP, only reachable within cluster        │
│  - Use for: service-to-service communication                 │
│  - my-api.default.svc.cluster.local → 10.96.45.12           │
│    → load balances to Pod IPs                                │
│                                                              │
│  90% of your Services will be ClusterIP.                     │
│  Your Java microservice calling another service:             │
│    RestTemplate("http://user-service:8080/api/users")        │
│    K8s DNS resolves "user-service" to ClusterIP              │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│ NodePort                                                     │
│  - Extends ClusterIP                                         │
│  - Opens a port (30000-32767) on EVERY node                  │
│  - External: <any-node-ip>:31080 → ClusterIP → Pods         │
│  - Use for: dev/test external access (not production)        │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│ LoadBalancer                                                 │
│  - Extends NodePort                                          │
│  - Provisions a cloud load balancer (AWS ELB, GCP LB)        │
│  - Gets a public IP                                          │
│  - Use for: production external traffic (one LB per service) │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│ Headless (ClusterIP: None)                                   │
│  - No virtual IP assigned                                    │
```

```
│  - DNS returns individual Pod IPs directly                    │
│  - Use for: StatefulSets, client-side load balancing         │
│  - db-0.db-headless.default.svc.cluster.local → Pod 0's IP  │
└──────────────────────────────────────────────────────────────┘
```

4.4 Ingress

```
Problem: 10 Services = 10 LoadBalancers = 10 public IPs = expensive
Solution: Ingress — one entry point, route by host/path

Ingress = L7 (HTTP) routing rules
Ingress Controller = the engine that implements them (NGINX, Traefik, ALB)

  Ingress Controller (1 cloud LB)
    → api.myapp.com/users   → user-service:8080
    → api.myapp.com/orders  → order-service:8080
    → web.myapp.com/        → frontend:3000

Features (depending on controller):
  - Path-based routing
  - Host-based routing
  - TLS termination (HTTPS)
  - Rate limiting
  - Rewrites, redirects
  - Auth (basic, OAuth)

Important: Ingress resource alone does nothing.
You MUST install an Ingress Controller in the cluster.
  - NGINX Ingress Controller (most common)
  - Traefik
  - AWS ALB Ingress Controller
  - Istio Gateway (if using Istio)
```

4.5 DNS (CoreDNS)

```
CoreDNS runs as a Deployment in kube-system.
Every Pod gets /etc/resolv.conf pointing to CoreDNS.

Resolution:
  Same namespace:      "my-service"         → my-
service.default.svc.cluster.local
  Cross namespace:     "my-service.staging"  → my-
```

```
service.staging.svc.cluster.local
  StatefulSet Pod:     "db-0.db-headless"    → specific Pod IP
  External:            "google.com"          → forwarded to upstream DNS

Why this matters for Java:
  In Spring app.properties:
    spring.datasource.url=jdbc:postgresql://postgres-service:5432/mydb

  "postgres-service" resolves via CoreDNS to the Service ClusterIP.
  Works because of the search domain in /etc/resolv.conf.
```

4.6 Network Policies

```
Firewall rules at Pod level.
By default: ALL Pods can talk to ALL Pods (open network).
When you apply a NetworkPolicy: denied by default, allow explicitly.

Example: Only frontend can reach backend

  apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
    name: backend-allow-frontend
  spec:
    podSelector:
      matchLabels:
        app: backend
    ingress:
      - from:
          - podSelector:
              matchLabels:
                app: frontend
        ports:
          - port: 8080

  Result:
    frontend → backend:8080  ✅
    anything else → backend  ❌

Requires a CNI that supports it: Calico, Cilium, Weave.
Flannel does NOT enforce Network Policies.
```

5. Storage — Persistence in K8s

5.1 The Problem

```
Containers are ephemeral. When a container restarts, its filesystem is lost.
For databases, file uploads, logs — you need persistent storage.
```

5.2 Volumes, PV, PVC, StorageClass

```
Volume Types (in-Pod, tied to Pod lifecycle):
  emptyDir:    temp directory, shared between containers in a Pod
               deleted when Pod dies. Use for: scratch space, sidecar sharing.

  hostPath:    maps a directory from the host node into the Pod
               dangerous in production (ties Pod to specific node).

  configMap:   mounts ConfigMap data as files
  secret:      mounts Secret data as files

Persistent Volumes (PV):
  - Cluster-level storage resource (not namespaced)
  - Backed by real storage: EBS, NFS, GCE PD, Azure Disk
  - Lifecycle independent of Pods

Persistent Volume Claims (PVC):
  - A REQUEST for storage by a Pod/user
  - Namespaced resource
  - Specifies: size, access mode, StorageClass
  - K8s binds PVC to a matching PV

StorageClass:
  - Defines HOW to dynamically provision storage
  - Eliminates manual PV creation

Flow:

  DYNAMIC PROVISIONING (standard):
    1. Admin creates StorageClass:
         provisioner: ebs.csi.aws.com
         parameters:
           type: gp3
    2. Developer creates PVC:
```

```
        storageClassName: gp3
        resources.requests.storage: 10Gi
    3. K8s calls the CSI provisioner → creates EBS volume
    4. PV auto-created and bound to PVC
    5. Pod mounts PVC

  STATIC PROVISIONING:
    1. Admin manually creates PV (pointing to existing storage)
    2. Developer creates PVC
    3. K8s binds PVC to PV

Access Modes:
  ReadWriteOnce (RWO):  mount read-write on ONE node
  ReadOnlyMany  (ROX):  mount read-only on MANY nodes
  ReadWriteMany (RWX):  mount read-write on MANY nodes (NFS, EFS)

Reclaim Policies:
  Retain:  keep PV after PVC deletion (manual cleanup)
  Delete:  delete PV and storage when PVC is deleted
```

5.3 StatefulSet Storage

```
StatefulSets use volumeClaimTemplates — each Pod gets its OWN PVC.

  StatefulSet: postgres (replicas: 3)
    volumeClaimTemplates:
      - name: data
        storage: 50Gi

  Creates:
    Pod postgres-0 → PVC data-postgres-0 → PV (50Gi)
    Pod postgres-1 → PVC data-postgres-1 → PV (50Gi)
    Pod postgres-2 → PVC data-postgres-2 → PV (50Gi)

  postgres-1 crashes → new postgres-1 Pod → re-mounts data-postgres-1
  PVCs survive Pod deletion (data is preserved).
```

---

6. Configuration — ConfigMaps, Secrets, Env

6.1 ConfigMaps

```
Store non-sensitive configuration as key-value pairs.

Create:
  kubectl create configmap app-config \
    --from-literal=DB_HOST=postgres-service \
    --from-literal=LOG_LEVEL=INFO \
    --from-file=application.properties

Use in Pod:

  As env vars:
    env:
      - name: DB_HOST
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: DB_HOST

  As mounted files:
    volumes:
      - name: config
        configMap:
          name: app-config
    volumeMounts:
      - name: config
        mountPath: /etc/config

  Hot reload: File-mounted ConfigMaps auto-update (~60s).
              Env var ConfigMaps do NOT auto-update — Pod restart required.

Java relevance:
  Mount application.properties or application.yml as a ConfigMap volume:
    mountPath: /config/application.properties

  Spring Boot picks up external config from /config/ by default.
```

6.2 Secrets

```
Store sensitive data (passwords, tokens, TLS certs).

Important: Secrets are base64 ENCODED, not encrypted (by default).
  echo "mypassword" | base64  → bXlwYXNzd29yZA==
  Anyone with etcd access can read them unless encryption at rest is enabled.

Best practices:
  1. Enable encryption at rest for etcd
  2. Use external secret managers (Vault, AWS Secrets Manager)
     with External Secrets Operator or Vault Agent Injector
  3. Mount as files (not env vars) — env vars leak in logs and process listings
  4. RBAC: restrict who can read Secrets
  5. Audit logging for Secret access

Types:
  Opaque:                    generic key-value (default)
  kubernetes.io/tls:         TLS cert + key
  kubernetes.io/dockerconfigjson:  Docker registry credentials
  kubernetes.io/service-account-token:  (auto-generated SA token)
```

---

7. Scheduling — Where Pods Land

7.1 nodeSelector

```
Simplest node assignment. Pod spec includes labels that must match a node.

  nodeSelector:
    disktype: ssd

Pod only schedules on nodes labeled disktype=ssd.
No match → Pod stays Pending.
```

7.2 Node Affinity

```
More expressive than nodeSelector. Supports hard and soft rules.

  requiredDuringSchedulingIgnoredDuringExecution:   ← HARD: must match
```

```
  preferredDuringSchedulingIgnoredDuringExecution:  ← SOFT: prefer but not
required

Example: Must be in us-east-1a or 1b, prefer m5.xlarge:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: topology.kubernetes.io/zone
                operator: In
                values: ["us-east-1a", "us-east-1b"]
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 80
          preference:
            matchExpressions:
              - key: node.kubernetes.io/instance-type
                operator: In
                values: ["m5.xlarge"]
```

7.3 Pod Affinity & Anti-Affinity

```
Schedule Pods relative to OTHER Pods.

Pod Affinity: "schedule me NEAR Pods with label X"
  Use: co-locate web server with cache for low latency

Pod Anti-Affinity: "schedule me AWAY from Pods with label X"
  Use: spread replicas across nodes/zones for HA

Example: Don't put two replicas of my-api on the same node
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values: ["my-api"]
          topologyKey: kubernetes.io/hostname
```

7.4 Taints & Tolerations

```
Taints on NODES: repel Pods
Tolerations on PODS: allow scheduling on tainted nodes

Taint a node:
  kubectl taint nodes gpu-node-1 gpu=true:NoSchedule

Only Pods with this toleration can schedule there:
  tolerations:
    - key: "gpu"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"

Effects:
  NoSchedule:        don't schedule new Pods (existing stay)
  PreferNoSchedule:  try to avoid, but allow if needed
  NoExecute:         evict existing Pods that don't tolerate

Use cases:
  - Dedicated GPU node pools
  - Master/control-plane nodes (system taint prevents user Pods)
  - Draining nodes for maintenance
```

7.5 Topology Spread Constraints

```
Spread Pods evenly across zones, nodes, or any topology.

  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app: my-api

  With 3 replicas across 3 zones:
    Zone A: 1 Pod, Zone B: 1 Pod, Zone C: 1 Pod

  Better than podAntiAffinity for multi-zone HA.
```

---

8. Scaling — HPA, VPA, Cluster Autoscaler

8.1 Horizontal Pod Autoscaler (HPA)

```
Automatically adjusts Pod replica count based on metrics.

How it works:
  1. Metrics Server collects CPU/memory from kubelet (every 15s)
  2. HPA controller reads metrics every 15s
  3. Calculates: desiredReplicas = current × (currentMetric / targetMetric)
  4. Scales Deployment up or down

  Example: target CPU 50%, current CPU 80%, current 3 replicas
    desired = 3 × (80/50) = 4.8 → 5 replicas

Spec:
  apiVersion: autoscaling/v2
  kind: HorizontalPodAutoscaler
  spec:
    scaleTargetRef:
      kind: Deployment
      name: my-api
    minReplicas: 2
    maxReplicas: 20
    metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: 50

Custom metrics (advanced):
  - Requests per second (via Prometheus Adapter)
  - Queue depth
  - Any custom metric exposed through the Metrics API

Cooldown:
  Scale-up: immediate (default)
  Scale-down: 5 minute stabilization window (prevents flapping)

Prerequisite: Pods MUST have resource requests set.
HPA uses requests as the baseline for % calculation.
```

8.2 Vertical Pod Autoscaler (VPA)

```
Adjusts CPU/memory REQUESTS (not replicas) based on usage.

Modes:
  Off:        only recommends (no changes)
  Initial:    sets resources on Pod creation only
  Auto:       evicts and recreates Pods with new resources

Use case: You don't know the right resource requests.
  VPA observes, recommends, and applies.

Warning: VPA and HPA on the SAME metric (CPU) will conflict.
  Use HPA for scaling replicas + VPA for memory recommendations.
```

8.3 Cluster Autoscaler

```
Scales the NUMBER OF NODES (not Pods).

Works with HPA:
  Load ↑ → HPA creates more Pods → not enough node capacity →
  Pods stuck Pending → Cluster Autoscaler adds nodes

Scale-up:
  Phase 1: Detect unschedulable (Pending) Pods
  Phase 2: Simulate which node group can fit them
  Phase 3: Call cloud API → launch new node(s)
  Phase 4: Node boots, joins cluster (~1-3 min)
  Phase 5: Pending Pods scheduled on new node

Scale-down:
  Phase 1: Find underutilized nodes (<50% capacity for 10 min)
  Phase 2: Check PDBs, local storage, unevictable Pods
  Phase 3: Drain node (respecting PDBs and graceful termination)
  Phase 4: Terminate node

Key: Scale-up takes 1-3 minutes. For latency-sensitive workloads,
over-provision slightly (keep warm spare capacity).
```

---

9. Security — RBAC, Pod Security, Secrets

9.1 RBAC (Role-Based Access Control)

```
Controls WHO can do WHAT on WHICH resources.

4 objects:
  Role              → permissions in a NAMESPACE
  ClusterRole       → permissions CLUSTER-WIDE
  RoleBinding       → grants Role to a subject in a NAMESPACE
  ClusterRoleBinding → grants ClusterRole CLUSTER-WIDE

Subjects:
  User, Group, ServiceAccount

Example: CI/CD bot can manage Deployments in staging:

  Role (staging namespace):
    rules:
      - apiGroups: ["apps"]
        resources: ["deployments"]
        verbs: ["get", "list", "create", "update", "patch", "delete"]

  RoleBinding:
    subjects:
      - kind: ServiceAccount, name: ci-bot, namespace: staging
    roleRef:
      kind: Role, name: deployment-manager

Test: kubectl auth can-i create deployments \
        --as=system:serviceaccount:staging:ci-bot -n staging
```

9.2 ServiceAccounts

```
Identity for Pods (and CI/CD bots, controllers).

Every namespace has a "default" ServiceAccount.
Pods get the default SA unless you specify one.

Best practice: Create dedicated SAs with least privilege.
  Don't let every Pod use the default SA.
```

```
  spec:
    serviceAccountName: order-service-sa
    automountServiceAccountToken: false  ← if Pod doesn't need API access
```

9.3 Pod Security Standards

```
Three levels (replaced PodSecurityPolicy in K8s 1.25):

  Privileged:  unrestricted (for system Pods)
  Baseline:    prevents obvious escalations (no hostNetwork, no privileged
containers)
  Restricted:  hardened (must run non-root, drop caps, read-only fs)

Enforce at namespace level:
  kubectl label namespace production \
    pod-security.kubernetes.io/enforce=restricted

Pod-level security context:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    readOnlyRootFilesystem: true
    allowPrivilegeEscalation: false
    capabilities:
      drop: ["ALL"]
```

---

10. Observability — Probes, Logs, Monitoring

10.1 Health Probes

```
┌──────────┬──────────────────────────┬──────────────────────────┐
│ Probe    │ Purpose                  │ On failure               │
├──────────┼──────────────────────────┼──────────────────────────┤
│ Liveness │ Is container alive?      │ RESTART container        │
│ Readiness│ Can it accept traffic?   │ Remove from Service      │
│ Startup  │ Has it finished booting? │ RESTART (after timeout)  │
└──────────┴──────────────────────────┴──────────────────────────┘

Probe types:
  httpGet:   GET /healthz → expect 200-399
```

```
  tcpSocket: connect to port → expect success
  exec:      run command → expect exit code 0

Java / Spring Boot integration:
  Spring Actuator provides:
    /actuator/health/liveness   → liveness probe
    /actuator/health/readiness  → readiness probe

  In deployment.yaml:
    livenessProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20

    readinessProbe:
      httpGet:
        path: /actuator/health/readiness
        port: 8080
      initialDelaySeconds: 10
      periodSeconds: 10

    startupProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
      failureThreshold: 30
      periodSeconds: 2
      # gives Spring Boot up to 60s to start
```

10.2 Logging

```
K8s does NOT provide built-in log aggregation. You need a stack:

Common: EFK (Elasticsearch + Fluentd + Kibana)
  - Fluentd DaemonSet on every node → collects container logs
  - Ships to Elasticsearch
  - Kibana for visualization

Or: ELK, Loki + Grafana, Datadog, CloudWatch

Container logs:
  Written to stdout/stderr → captured by container runtime →
```

```
  stored at /var/log/containers/ on the node →
  collected by log shipper (Fluentd/Filebeat)

Java relevance:
  Configure your Java app to log to STDOUT (not files).
  SLF4J + Logback with ConsoleAppender.
  K8s captures stdout automatically.

  In logback-spring.xml:
    <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
      <encoder>
        <pattern>%d{ISO8601} [%thread] %-5level %logger - %msg%n</pattern>
      </encoder>
    </appender>
```

10.3 Monitoring

```
Common stack: Prometheus + Grafana

Prometheus:
  - Scrapes /metrics endpoints from Pods
  - Time-series database
  - Alert rules (Alertmanager)

Grafana:
  - Dashboards, visualization

Metrics Server:
  - Lightweight, in-cluster resource metrics (CPU/memory)
  - Required for HPA and kubectl top
  - NOT for long-term monitoring (no history)

Java relevance:
  Micrometer (Spring Boot Actuator) exposes Prometheus metrics:
    /actuator/prometheus

  In pom.xml:
    micrometer-registry-prometheus

  Prometheus scrapes this endpoint automatically if annotations are set:
    annotations:
      prometheus.io/scrape: "true"
```

```
      prometheus.io/port: "8080"
      prometheus.io/path: "/actuator/prometheus"
```

---

11. Deployment Strategies — Zero Downtime

11.1 Rolling Update (Built-in)

```
Default strategy. K8s creates new Pods before killing old ones.

Requirements for zero downtime:
  1. maxUnavailable: 0 (don't kill old before new is ready)
  2. Readiness probe (K8s waits for new Pod to be ready)
  3. Graceful shutdown (handle SIGTERM, drain connections)
  4. preStop hook: sleep 5 (handles kube-proxy iptables race)
  5. PodDisruptionBudget (protects during node drains)

  Flow:
    v1-a ✅, v1-b ✅, v1-c ✅
    v2-d created → passes readiness → v1-a terminated
    v2-e created → passes readiness → v1-b terminated
    v2-f created → passes readiness → v1-c terminated
    v2-d ✅, v2-e ✅, v2-f ✅
```

11.2 Blue-Green

```
Not built into K8s natively. Implemented via Service label switching.

  Step 1: "blue" Deployment running (v1), Service points to blue
    Service selector: version=blue

  Step 2: Deploy "green" Deployment (v2), same label app=my-api, version=green
    Green Pods are running but receive NO traffic

  Step 3: Test green independently (internal endpoint, smoke tests)

  Step 4: Switch Service selector: version=green
    All traffic instantly switches to green

  Step 5: If issues → switch back to version=blue (instant rollback)
```

```
  Pros: instant cutover, instant rollback
  Cons: double the resources during transition
```

11.3 Canary

```
Route small % of traffic to new version, verify, then roll out.

With Ingress annotation (NGINX):
  nginx.ingress.kubernetes.io/canary: "true"
  nginx.ingress.kubernetes.io/canary-weight: "10"

  10% of traffic → v2 canary Pods
  90% of traffic → v1 stable Pods

  Monitor error rates, latency for canary.
  If good → increase to 50% → 100%.
  If bad → remove canary annotation (instant rollback).

Advanced: Istio VirtualService for traffic splitting.
  Or Argo Rollouts for automated canary with metric analysis.
```

---

12. Helm — Package Management

```
Helm = package manager for Kubernetes (like apt/npm for K8s manifests).

Concepts:
  Chart:    a package of K8s manifests (templates + values)
  Release:  an installed instance of a Chart
  Values:   configuration that customizes the Chart
  Repository: a collection of Charts (like Docker Hub for Helm)

Common operations:
  helm repo add bitnami https://charts.bitnami.com/bitnami
  helm install my-postgres bitnami/postgresql --values values.yaml
  helm upgrade my-postgres bitnami/postgresql --values values.yaml
  helm rollback my-postgres 1
  helm uninstall my-postgres

Chart structure:
  mychart/
```

```
    Chart.yaml          ← metadata (name, version)
    values.yaml         ← default configuration
    templates/
      deployment.yaml   ← Go template: {{ .Values.replicaCount }}
      service.yaml
      ingress.yaml
      _helpers.tpl      ← template helpers

Why Helm matters for Java devs:
  - Package your microservice as a Helm chart
  - Same chart, different values per environment (dev/staging/prod)
  - CI/CD pipeline: helm upgrade --install in each stage
  - Install dependencies (PostgreSQL, Redis, Kafka) from public charts
```

---

13. Java on Kubernetes — JVM-Specific Concerns

This section is critical. The JVM has behaviors that interact poorly with K8s defaults.

13.1 Memory: Container Limits vs JVM Heap

```
THE PROBLEM:
  Container limit: 512Mi
  JVM default heap: tries to use 25% of PHYSICAL machine memory
  Machine has 64GB RAM → JVM wants 16GB heap → OOMKilled instantly

THE FIX (Java 10+):
  JVM flag: -XX:+UseContainerSupport (enabled by default since Java 10)
  JVM sees container limits, not host memory.

  With 512Mi container limit:
    JVM sees 512Mi as total memory
    Default MaxRAMPercentage = 25% → ~128Mi heap
    You probably want more → set explicitly:

  JAVA_OPTS: "-XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0"

  This gives ~384Mi heap out of 512Mi container limit.
  Remaining ~128Mi for: metaspace, thread stacks, native memory, off-heap.

  Rule of thumb for container memory limit:
```

```
    container_limit = max_heap + metaspace(~256Mi) + thread_stacks(threads × 1Mi)
+ buffer

MEMORY BUDGET EXAMPLE:
  Container limit: 1Gi (1024Mi)
    Max heap:        700Mi  (-XX:MaxRAMPercentage=68.0)
    Metaspace:       ~150Mi (-XX:MaxMetaspaceSize=150m)
    Thread stacks:   ~100Mi (100 threads × 1Mi)
    Native/buffers:  ~74Mi
    Total:           1024Mi ✅
```

13.2 CPU: Container Limits vs JVM Threads

```
THE PROBLEM:
  CPU limit: 1000m (1 core)
  JVM's Runtime.availableProcessors() might return the HOST's core count
  (fixed in modern Java, but older apps or libs may misbehave)

  ForkJoinPool.commonPool() sizes to availableProcessors()
  If it sees 32 cores but only has 1 → thread oversubscription → throttling

THE FIX:
  Java 11+: -XX:+UseContainerSupport (default) → sees container CPU limit
  Java 8u191+: same flag available

  Verify in container:
    Runtime.getRuntime().availableProcessors()  → should return ~1 for 1000m

  For 1000m (1 core): JVM sees 1 processor
  For 2000m (2 cores): JVM sees 2 processors
  For 500m (0.5 cores): JVM sees 1 processor (minimum 1)

CPU REQUESTS vs LIMITS:
  - Requests: guaranteed CPU time (scheduler uses this)
  - Limits: maximum — container is THROTTLED (not killed) if exceeded
  - Many teams set NO CPU limit (only requests) to avoid throttling
    → Pod can burst when needed, requests guarantee minimum
```

13.3 Startup Time

```
Java / Spring Boot apps are SLOW to start (10-60 seconds).
```

```
Problems without proper config:
  - Liveness probe fires before app is ready → restarts → CrashLoopBackOff
  - Readiness probe fails → traffic never routes → 503 errors
  - HPA scales up but new Pods take 30s → not helpful for spikes

Solutions:

  1. Startup Probe:
    startupProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
      failureThreshold: 30
      periodSeconds: 2
    # Gives up to 60s (30 × 2s) before liveness takes over

  2. Reduce startup time:
    - Use Spring Boot lazy initialization: spring.main.lazy-initialization=true
    - Use CDS (Class Data Sharing) for faster class loading
    - Use GraalVM native images (Quarkus/Micronaut) for <1s startup
    - Pre-compile: -XX:+TieredCompilation -XX:TieredStopAtLevel=1

  3. Graceful shutdown in Spring Boot:
    server.shutdown=graceful
    spring.lifecycle.timeout-per-shutdown-phase=30s

    K8s sends SIGTERM → Spring stops accepting new requests →
    finishes in-flight requests → exits cleanly within 30s
```

13.4 Dockerfile Best Practices for Java

```
# Multi-stage build — small final image
FROM eclipse-temurin:21-jdk AS builder
WORKDIR /app
COPY . .
RUN ./mvnw package -DskipTests

FROM eclipse-temurin:21-jre          # JRE only, not JDK
WORKDIR /app
COPY --from=builder /app/target/myapp.jar app.jar

# Non-root user (K8s security context)
RUN addgroup --system appgroup && adduser --system appuser --ingroup appgroup
USER appuser
```

```
EXPOSE 8080

ENTRYPOINT ["java", \
  "-XX:MaxRAMPercentage=75.0", \
  "-XX:+UseContainerSupport", \
  "-jar", "app.jar"]
```

```
Key points:
  - Multi-stage: build with JDK, run with JRE (smaller image)
  - eclipse-temurin: trusted OpenJDK distribution
  - Non-root user: matches K8s runAsNonRoot
  - Container-aware JVM flags baked in
  - No :latest tag — pin version
  - .dockerignore: exclude .git, target, node_modules
```

13.5 Spring Boot + K8s Integration

```
Spring Cloud Kubernetes:
  - Auto-discovers config from ConfigMaps and Secrets
  - Service discovery via K8s Services (no Eureka needed in K8s)
  - Config reload: watch ConfigMap changes, refresh beans

Dependencies:
  spring-cloud-starter-kubernetes-client-config
  spring-cloud-starter-kubernetes-client-all

Actuator endpoints for K8s probes:
  management.endpoint.health.probes.enabled=true
  management.health.livenessState.enabled=true
  management.health.readinessState.enabled=true

  → /actuator/health/liveness
  → /actuator/health/readiness

Profile activation:
  In ConfigMap or env:
    SPRING_PROFILES_ACTIVE=kubernetes,production
```

---

14. Namespaces, Resource Quotas & Limit Ranges

14.1 Namespaces

```
Virtual clusters within a physical cluster. Isolation boundary.

Default namespaces:
  default:      where resources go if you don't specify
  kube-system:  control plane components (CoreDNS, kube-proxy, etc.)
  kube-public:  readable by all (rarely used)
  kube-node-lease: node heartbeat leases

Use cases:
  Per-environment: dev, staging, production
  Per-team: team-alpha, team-beta
  Per-application: payment-system, user-system

Things that are namespaced: Pods, Services, Deployments, ConfigMaps, Secrets,
PVCs
Things that are NOT: Nodes, PVs, StorageClasses, ClusterRoles, Namespaces
```

14.2 Resource Quotas

```
Limit total resources a namespace can consume.

apiVersion: v1
kind: ResourceQuota
metadata:
  name: staging-quota
  namespace: staging
spec:
  hard:
    requests.cpu: "10"          # total CPU requests across all Pods
    requests.memory: 20Gi       # total memory requests
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"                  # max 50 Pods in this namespace
    services.loadbalancers: "2" # max 2 LoadBalancer Services

If a new Pod would exceed the quota → rejected by API server.
```

14.3 Limit Ranges

```
Default and enforce limits for individual Pods/containers in a namespace.

apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: staging
spec:
  limits:
    - type: Container
      default:           # applied if Pod doesn't set limits
        cpu: "500m"
        memory: "256Mi"
      defaultRequest:    # applied if Pod doesn't set requests
        cpu: "100m"
        memory: "128Mi"
      max:               # hard cap per container
        cpu: "2"
        memory: "2Gi"
      min:               # minimum per container
        cpu: "50m"
        memory: "64Mi"
```

15. CRDs & Operators

```
CRD (Custom Resource Definition):
  Extend the K8s API with your own resource types.
  After defining a CRD, you can: kubectl get mydatabases

Operator:
  A controller that watches CRDs and takes action.
  Encodes human operational knowledge into code.

  Example: PostgreSQL Operator
    - You create: kind: PostgresCluster, replicas: 3
    - Operator automatically:
        Creates StatefulSet, Services, PVCs
        Configures replication
        Sets up automated backups
```

```
        Handles failover
        Manages upgrades

Popular operators:
  Prometheus Operator     → monitoring
  Strimzi                → Kafka on K8s
  Cert-Manager           → TLS certificate management
  Zalando Postgres       → PostgreSQL HA
  Elastic Cloud on K8s   → Elasticsearch

CRD + Operator = the K8s-native way to manage complex stateful systems.
```

---

16. Service Mesh (Istio Basics)

```
A service mesh adds infrastructure-level networking features
without changing application code.

What Istio adds:
  - mTLS between all services (encryption in transit)
  - Traffic management (canary, blue-green, fault injection)
  - Observability (distributed tracing, metrics, access logs)
  - Circuit breaking, retries, timeouts
  - Rate limiting

How it works:
  Istio injects an Envoy sidecar proxy into every Pod (via mutating webhook).

  Pod: [my-java-app] ←→ [envoy-proxy] → network → [envoy-proxy] ←→ [target-app]

  All traffic goes through Envoy. Istio control plane configures all Envoys.

Key objects:
  VirtualService: traffic routing rules (canary weights, header-based routing)
  DestinationRule: circuit breakers, connection pool settings
  Gateway: ingress traffic configuration
  PeerAuthentication: mTLS policies

When to use:
  - 20+ microservices needing consistent security/observability
  - Zero-trust networking requirements
  - Advanced traffic management (gradual rollouts)
```

```
When NOT to use:
  - Small clusters (< 10 services) — overhead not worth it
  - Performance-critical with tight latency budgets (Envoy adds ~1-3ms per hop)
```

---

17. CI/CD with Kubernetes

```
Typical pipeline for a Java microservice:

  1. Developer pushes to Git
  2. CI (Jenkins/GitHub Actions/GitLab CI):
     a. Compile: mvn clean package
     b. Test: mvn test + integration tests
     c. Build image: docker build -t myregistry/my-app:v1.2.3
     d. Push image: docker push myregistry/my-app:v1.2.3
     e. Update K8s manifest or Helm values with new image tag

  3. CD (ArgoCD / Flux / Helm):
     a. Detect new image tag in Git (GitOps)
     b. helm upgrade --install my-app ./chart --set image.tag=v1.2.3
     c. K8s rolling update begins
     d. Monitor rollout: kubectl rollout status deployment/my-app
     e. If fails → automatic rollback (or manual: helm rollback)

GitOps approach (ArgoCD):
  - K8s manifests stored in Git (source of truth)
  - ArgoCD watches Git repo
  - Any change in Git → ArgoCD syncs to cluster
  - Drift detection: if someone manually changes cluster, ArgoCD reverts

Image tag strategy:
  - Never use :latest in production
  - Use Git SHA: myapp:abc123f
  - Or semver: myapp:v1.2.3
  - Enables deterministic rollbacks
```

---

18. Troubleshooting Playbook

Pod stuck in Pending

```
kubectl describe pod <name>  → check Events

Common causes:
  "Insufficient cpu/memory"          → cluster full, add nodes
  "Didn't match node selector"       → wrong labels on nodes
  "Taint not tolerated"              → add toleration or remove taint
  "PVC not bound"                    → create PVC, check StorageClass
  "ResourceQuota exceeded"           → quota limit hit in namespace
```

Pod in CrashLoopBackOff

```
kubectl logs <pod> --previous       → logs from crashed container
kubectl describe pod <pod>          → exit code, events

Common causes:
  Exit 1:    app error (check logs, config, dependencies)
  Exit 137:  OOMKilled (increase memory limit)
  Exit 139:  Segfault
  Liveness probe killing healthy but slow container → tune probe timing
```

Pod in ImagePullBackOff

```
kubectl describe pod <pod>  → look for image pull error

Common causes:
  Wrong image name/tag
  Private registry — missing imagePullSecrets
  Rate limiting (Docker Hub anonymous pull limit)
```

Service not reachable

```
Step 1: kubectl get endpoints <service>
  → Are there Pod IPs listed? If empty → selector doesn't match Pod labels

Step 2: kubectl exec -it <pod> -- curl <service>:<port>
```

```
  → Test from inside the cluster

Step 3: Check readiness probes
  → If Pods aren't ready, they're removed from endpoints

Step 4: Check Network Policies
  → Might be blocking traffic
```

Node NotReady

```
kubectl describe node <node>  → check Conditions and Events

Common causes:
  kubelet not running           → SSH into node, systemctl status kubelet
  Disk pressure                 → node storage full
  Memory pressure               → node OOM
  Network partition             → node can't reach API server
  Certificate expired           → kubelet certs need renewal
```

---

19. Essential Commands Cheat Sheet

```
# ──────── CLUSTER INFO ────────
kubectl cluster-info
kubectl get nodes -o wide
kubectl version --short
kubectl api-resources                      # list all resource types

# ──────── NAMESPACE ────────
kubectl get namespaces
kubectl config set-context --current --namespace=staging  # switch default ns
kubectl get all -n kube-system

# ──────── WORKLOADS ────────
kubectl get pods -o wide                   # see node placement and IPs
kubectl get pods --all-namespaces          # all namespaces
kubectl get deployments,replicasets,pods
kubectl describe pod <pod>                 # detailed info + events
kubectl logs <pod>                         # container logs
kubectl logs <pod> --previous              # previous (crashed) container logs
kubectl logs <pod> -c <container>          # specific container in multi-
```

```
container Pod
kubectl exec -it <pod> -- /bin/sh          # shell into container
kubectl port-forward <pod> 8080:8080       # access Pod locally

# ──────── DEPLOYMENTS ────────
kubectl rollout status deployment/<name>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>
kubectl rollout undo deployment/<name> --to-revision=2
kubectl set image deployment/<name> app=image:tag
kubectl scale deployment/<name> --replicas=5

# ──────── DEBUGGING ────────
kubectl get events --sort-by='.lastTimestamp'
kubectl describe nodes | grep -A 10 "Allocated resources"
kubectl top nodes
kubectl top pods
kubectl debug -it <pod> --image=busybox    # ephemeral debug container
kubectl run debug --image=busybox -it --rm -- sh  # temp debug Pod

# ──────── SERVICES & NETWORKING ────────
kubectl get svc
kubectl get endpoints <service>
kubectl get ingress
kubectl get networkpolicies

# ──────── CONFIG & SECRETS ────────
kubectl get configmaps
kubectl get secrets
kubectl create configmap <name> --from-file=app.properties
kubectl create secret generic <name> --from-literal=password=abc123

# ──────── STORAGE ────────
kubectl get pv
kubectl get pvc
kubectl get storageclass

# ──────── RBAC ────────
kubectl auth can-i create deployments --as=<user>
kubectl auth can-i '*' '*'                 # am I cluster admin?
kubectl get roles,rolebindings -n <ns>
kubectl get clusterroles,clusterrolebindings

# ──────── NODE MANAGEMENT ────────
kubectl cordon <node>                      # mark unschedulable
```

```
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>                    # mark schedulable
kubectl taint nodes <node> key=value:NoSchedule
kubectl taint nodes <node> key=value:NoSchedule-  # remove taint

# ──────── SCALING ────────
kubectl autoscale deployment/<name> --min=2 --max=10 --cpu-percent=60
kubectl get hpa

# ──────── HELM ────────
helm repo add <name> <url>
helm search repo <keyword>
helm install <release> <chart> --values values.yaml
helm upgrade <release> <chart> --values values.yaml
helm rollback <release> <revision>
helm list
helm uninstall <release>
```

Quick Reference: Object Hierarchy

```
Cluster
  ├── Nodes (worker machines)
  ├── Namespaces
  │     ├── Deployments → ReplicaSets → Pods → Containers
  │     ├── StatefulSets → Pods (ordered, with PVCs)
  │     ├── DaemonSets → Pods (one per node)
  │     ├── Jobs / CronJobs → Pods (run to completion)
  │     ├── Services (ClusterIP / NodePort / LoadBalancer / Headless)
  │     ├── Ingresses (L7 routing)
  │     ├── ConfigMaps, Secrets
  │     ├── PVCs → bound to PVs
  │     ├── ServiceAccounts, Roles, RoleBindings
  │     ├── NetworkPolicies
  │     ├── HPA, VPA
  │     └── PodDisruptionBudgets
  ├── PersistentVolumes (cluster-scoped)
  ├── StorageClasses (cluster-scoped)
  ├── ClusterRoles, ClusterRoleBindings (cluster-scoped)
  └── CRDs (cluster-scoped)
```